DSAII- HW10

Tomas Salaz

April 13, 2026

Problem 1

When using the Union-Find data structure with rank, If an object x is not the set leader, then $rank(x) < rank(x.leader)$. For a set X, $size(X) \geq 2^{rank(x.leader)}$. Since there are n objects the highest possible rank is $O(log\ n)$, and only leaders can change rank. There are also at most $n/2^r$ objects with rank r.

If there are m `MakeSet, Union, and FindSet` operations, n of them are `MakeSet` the total cost of all operations is $\Theta(m\ log\ n)$

Proof by Induction on rank k of tree T
BC: $k = 0$ implies that a tree has a single root node, so $k = 2^0 = 1$ thus the root node has rank 0. **IH**: For all $j < k$, a tree T of rank j contains at least 2^j nodes.
IS: A tree of rank k is only created when 2 Trees of rank $k-1$ are unified, `T = Union(`T_1, T_2`)`. By the IH, each subtree has at least 2^{k-1} nodes, so T has $2^{k-1} + 2^{k-1} = 2^k$ nodes.

The Proof above implies that is we have n total nodes and $2^k \leq n$, then $k \leq log\ n$, thus the maximum rank of any tree is at most $log\ n$.
Without Path Compression, Rank=Height for any tree T, therefore $Height(T) \leq log\ n$. Thus any `FindSet` operation is also bounded by the height. `MakeSet and Union` each cost $O(1)$, so over m operations the total cost in the worst case is $\Theta(m\ log\ n)$.

Consider a sequence of `FindSet` operations an a node at the max depth. Since the Height od the tree is bounded by $\Theta(log\ n)$ and each operatoin takes $\Theta(log\ n)$ time, performing m operations costs a total of $\Theta(m\ log\ n)$ thus proving the bound.

Problem 2

The algorithm Takes in a Connected Weighted Graph G with weight function $w : E \to R$and Outputs a Minimum-weight Cycle Cover, F where $F \subset E$.

```
Algo Begin
    1. Find Max-Weight Span Tree, T, of G. Using Kruskal's Algo and
    with edges sorted in decreasing weight.

    2. Return MaxSpan Tree Edges, F
    F = E-T (Elements in Edge Set that are NOT in Max-Span
Algo End
```

For Step 1, Sort all edges E in decreasing order. Init T, the Max-Weight Span Tree, to be Empty and init a `Union-Find` with each vertex in its own set.
For each edge (u, v): if `FIND(u)` $\neq$ `FIND(v)` then `UNION(u,v)` to add to T. Stop When T has $n - 1$ edges.

Correctness

We must show 2 things: (1). F is a Cycle Cover and (2) F has min total weight among all Cycle Covers.
(1). The construction of T is acyclic and connects all vertices. Removing F where $F = (E - T)$ from G leaves the edges in T that form a Tree. Since T is spanning and acyclic, any cycle in G must contain at lease one edge not in T. Therefore, every cycle cove contains at least one edge in $F = E - T$, so F is a cycle cover.
(2). Let F' be any Cycle Cover for G, then $E - F'$ is also acyclic, and since G is connected then $E - F'$ must be a subgraph that contains some spanning tree. we also know that $Weight(E - F') \leq Weight(T')$ where T' is the max weight spanning tree of the subgraph $E - F'$. T' is also a spanning tree of G. Thus, $Weight(F') = Weight(E - F') \geq Weight(E) - Weight(T) = Weight(F)$
So $Weight(F) \leq Weight(F')$ for any cycle cover F' thus F is the min weight.

Run Time

Sorting the edges by decreasing weight take $O(m\ log\ m) = O(m\ log\ n)$ time since $m \leq n^2$.
The main loop for Kruskal's has m iterations, each doing 2 `FIND()` operations and at most 1 `UNION()`. With Union by Rank and Path compression, the Run-time is $O(m\ log^* n) = O(m)$.
Computing $F = E - T$ iterates through all m edges and checks membership in T, Using a Hash Set for T, this has $O(m)$ runtime.
The Total runtime is $O(m\ log\ n)$

Problem 3

Proof by Counter Example
Let $G = (V, E)$ with $V = a, b, c, d$ and the weight of the edges be $(a, b) = 5, (a, c) = 1$, and $(a, d) = 3$.
G is a tree since it is Connected and Acyclic, so there must be a unique Spanning Tree. This tree is simple and its $MST = \{(a, b), (a, c), (a, d)\}$.
Let $A = \{(a, b)\}$. Since (a, b) is an element of T, A is included in the MST of G.
Let $S = \{a, b\}$ and $V - S = \{c, d\}$. The Cut $(S, V - S)$ respects A because the only edge in A is (a, b) and both its endpoints are within S, so they do not cross the Cut.
The only edges crossing the cut are (a, c) and (a, d), so the Light Edge of the cut is (a, c) since the weight is 1.
A Safe Edge implies that a MST exists and contains $A \bigcup \{(a, d)\} = \{(a, b), (a, d)\}$.
The unique MST, T, contains both edges so (a, d) is safe, but is not the Light Edge for the cut.
Thus, a Graph G, a set $A \subset MST$, a cut $(S, V - S)$ and a crossing cut safe edge (a, d) is not the lightest edge in the crossing cut, Therefore the converse of the safe edge theorem is false.

Problem 4

If an edge (u, v) is in some Min-Span-Tree for graph G the (u, v) is a light edge crossing some cut in G
Proof
Let T be the MST of G and contains the edge (u, v). To construct a cut, we remove the edge (u, v) from T. Since T is a tree, removing any edge disconnects it into 2 connected components.
Let S be the vertex set containing the component u and let $V - S$ be the vertex set containing v.
The partition $(S, V - S)$ defined the cut of G, and by construction if, u is an element in S and v is an element in $V - S$, so (u, v) crosses the cut.

To show that (u, v) is a light edge, we can prove this using contradiction.
Suppose (u, v) is NOT a light edge foe the cut $(S, V - S)$, Then another crossing edge, (x, y) exists that is lighter, $Weight(x, y) < Weight(u, v)$ and x is element in S and y is an element in $V - S$.
Consider the edge set, $T' = (T - \{(u, v)\} \bigcup \{(x, y)\}$.
If T' is a Spanning Tree of G, then T' connects all vertices, So removing (u, v) and replacing with (x, y) reconnects them since u and x are in the same set AND v and y are in the same set. Thus T' has $n - 1$ edges and remains connected, so it is still spanning tree.
Since
$Weight(T') = Weight(T) - Weight(u, v) + Weight(x, y)$

and
$Weight(x, y) < Weight(u, v)$
Then T' is a spanning tree with a smaller total weight than T, thus contradicting the assumption that T is a Min-Span-Tree.